Lecture 1

1.整 数 类 型

byte 大小 8 bit 取值范围 -128 to 127

short 大小 16 bit 取值范围 -32,768 to 32,767 （-2^15~2^15-1）

int 大小 32 bit 取值范围 -2^ 31 to 2^31 - 1

long 大小 64 bit 取值范围 -2^ 63 to 2^63 - 1

2.char 类型只能存放单个字符，比如

char ch = ‘a’;

只能使用单引号

字符串 类型需要使用双引号，比如

String s = “aa”;

字符串其实是由字符加起来的。

3.ASCII is an encoding scheme that represents each character with a **7-bit** value.

4.Integrated Development Environment (IDE)

Source Code Editor(源代码编辑器),Compiler/Interpreter(编译器/解释器),Build Automation Tools(构建自动化工具),Debugger（调试器）

5. 转义： \

\" -> "

\' -> '

\\ -> \

System.out.println("\"Hello World\"");

Lecture 2 Data Types, Math, Client Input Debugging

1.数据类型

Data Type	Values	Sample Operators	Sample Values
int	integers / natural numbers	+ - * / %	25 -1 2147483647
double	floating-point numbers	+ - * /	2.5 -1.0 1.7e308
boolean	boolean values	&& || !	true false
char	characters		'A' '\n' '&' '2'
String	sequences of characters	+	"Hello" "a \n" "2.5"

小 数 类 型

float 大小 32 bit 取值范围 +-10^38

double 大小 64 bit 取值范围 +-10^308

这些都可能有点不准确 可能会出现舍入错误 0.8 可能变成 0.7999999999

2.Integer Division 5/2=2

Integer Division by 0 ： runtime error

Compile Error (编译错误)：代码语法错误

Runtime error(运行时错误)：除以零 数组越界 空指针异常内存溢出

Remainder：5%2 = 1

正数 / 0.0 = Infinity (正无穷)

负数 / 0.0 = -Infinity (负无穷)

Math.sqrt(-1.0) ， **0.0 / 0.0 = NaN (Not a Number)**

3.

Expression	Value	Meaning
ex++	6	postfix increment (add by 1)
++ex	6	prefix increment
ex--	4	postfix decrement (minus 1)
--ex	4	prefix decrement
ex += 2	12	ex = ex + 2
ex *= 10	50	ex = ex * 10

ex++: 后缀递增 0 先用后加

```
int ex = 5;
int y = ex++;  //y=5 ex=6
```

++ex:先加后用

```
int ex = 5;
int y=++ex;    //y=6 ex=6
```

Math Function	Meaning
`double abs(double a)`	absolute value of a
`double max(double a, double b)`	maximum of a and b
`double min(double a, double b)`	minimum of a and b
`double sin(double theta)`	sine of theta
`double cos(double theta)`	cosine of theta
`double tan(double theta)`	tangent of theta
`double toRadians(double degrees)`	convert angle from degrees to radians
`double toDegrees(double radians)`	convert angle from radians to degrees
`double exp(double a)`	exponential (e^a)
`double log(double a)`	natural log ($\log_e a$, or ln a)
`double pow(double a, double b)`	raise a to the b-th power (a^b)
`long round(double a)`	round a to the nearest integer
`double random()`	random number in [0, 1)
`double sqrt(double a)`	square root of a

4. math function 数学公式　Math.abs(-5.0)
平方根：Math.sqrt(num);　Math.asin(x);反正弦　Math.acos(x)　Math.atan(x)　pi: Math.PI
两个数的最大值 Math.max(num1,num2);
随机数 Math.random()

5. casting

We need to convert the long output to an integer first by casting

int i = **(int)** Math.round(3.14159);

6.Convert Strings to Numbers
String s = "5";
int i = Integer.parseInt(s);　　//　　Integer/Double: Library
double d = Double.parseDouble(t);　　// parseInt()/parseDouble():function

Interactive User Input

- Finally, we use scanner object to get user input with the following steps:

```
import java.util.Scanner;

public class DateToDay {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String input = scanner.nextLine();
        int y = Integer.parseInt(input);
        input = scanner.nextLine();
        int m = Integer.parseInt(input);
        input = scanner.nextLine();
        int d = Integer.parseInt(input);
        int a = y - (14 - m) / 12;
        int b = a + a/4 - a/100 + a/400;
        int c = m + 12 * ((14 - m) / 12) - 2;
        int day = (d + b + 31 * c / 12) % 7;
        System.out.println("It's day " + day + " !");
    }
}
```

need to import Scanner library

create a Scanner object, **one object is enough**!

call **nextLine()** to read one line of input and to return it as a String

convert the String to the correct data type (if necessary)

repeat nextLine() and type conversion for **each** *line* of input

Lab 2

$$r = \sqrt{x^2 + y^2}$$
$$\theta = \arctan\left(\frac{y}{x}\right)$$

1.键盘输入+数学公式

Input = kb.nextLine()

```
import java.util.Scanner;
public class Lab21 {
    public static void main(String[] args){
        Scanner kb=new Scanner(System.in);
        double x = Double.parseDouble(kb.nextLine());
        double y = Double.parseDouble(kb.nextLine());
        double r = Math.sqrt(x*x + y*y);
        double theta = Math.atan2(y,x);
        System.out.println("r = "+ r +"\ntheta = "+theta);
    }
}
```

用于象限 y,x 坐标

2.Math.round()

```
import java.util.Scanner;
public class Lab22{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        double cyan =Double.parseDouble(sc.nextLine());
        double magenta =Double.parseDouble(sc.nextLine());
        double yellow =Double.parseDouble(sc.nextLine());
        double black =Double.parseDouble(sc.nextLine());
        double white = 1-black;
        long red = Math.round(255*white*(1-cyan));
        long green = Math.round(255*white*(1-magenta));
        long blue = Math.round(255*white*(1-yellow));
        System.out.println("red = "+ red +"\ngreen = "+ green +"\nblue = "+ blue);
    }
}
```

3.Math.toRadians　将角度转化为弧度

2*r*Math.asin(Math.sqrt(Math.pow(Math.sin(Math.toRadians((x2-x1)/2)),2)+Math.cos(Math.toRadians(x1))*Math.cos(Math.toRadians(x2))*Math.pow(Math.sin(Math.toRadians((y2-y1)/2)),2)));

$$dist = 2r \arcsin\left(\sqrt{\sin^2\left(\frac{x_2 - x_1}{2}\right) + \cos x_1 \cos x_2 \sin^2\left(\frac{y_2 - y_1}{2}\right)}\right)$$

Lecture 3 Flow Controls

1.if-else

```
if (score < 40)
    System.out.println("Fail");
else {
    if (score < 50)
        System.out.println("Third Class");
    else {
        if (score < 60)
            System.out.println("Lower Second Class");
        else {
            if (score < 70)
                System.out.println("Upper Second Class");
            else
                System.out.println("First Class");
        }
    }
}
```

```
if(条件表达式1)
{
    //语句1
}else if(条件表达式2){
    //语句2
}else if(条件表达式3){
    //语句3
}else{
    //最后的语句
}
```

2.while 循环特别适用于你不确切知道需要循环多少次循环次数由某个条件决定

While(){

}

Lab3

1. Lab exe3.1 Amstrong 数
2. Lab 3 Challenge -Lab C – Hailstone Sequence Length （奇偶运行不同）
3. Exe31 1/n 级数求和
4. Exe32 距离 21

```
if (n1>21&&n2>21){
    System.out.println("-1");
}
else if (n1>21&&n2<21){
    System.out.println(n2);
}
else if (n1<21&&n2>21){
    System.out.println(n1);
}
else{
    System.out.println(Math.max(n1,n2));
}
```

5. Exe33 传染人数

6. 勾股定理 exe34

```
int max = Math.max(Math.max(n1, n2), n3);
int min = Math.min(Math.min(n1, n2), n3);
int mid = n1 + n2 + n3 - max - min;
```

7. 完美数 CW131

```
int max = Math.max(Math.max(n1, n2), n3);
int min = Math.min(Math.min(n1, n2), n3);
int mid = n1 + n2 + n3 - max - min;
```

Lec4 For Loops, Arrays, Static Methods 1

1. //for 循环：在确定循环次数后使用 初始化语句 条件判断 条件控制

2.随机掷色子

```
double rand = ((Math.random() * 6) + 1);

int dice = (int) rand;
```

3.数组初始化

Array initialization
When created, arrays are automatically **initialized**
with the default value of their type. E.g.

```
String[] s = new String[100];
// default values: null
boolean[] b = new boolean[4];
// default values: false
int[] i = new int[10];
// default values: 0
```

4 . Array Lengths

Arrays can not be resized after creation

We can get the length using **myArray.length**.

Ex:

```
String [] myArray = {"a", "b", "c", "d", "e"};
int len = myArray.length; //will return len = 5
boolean[] myArray = {true, false, true};
int len = myArray.length; //will return len=3
```

```
public static void main(String[] args) {
    int[] diceList = new int[4];
    for(int i = 0; i < 4; i++){
        double rand = ((Math.random() * 6) + 1);
        int dice = (int) rand;
        diceList[i] = dice;
        System.out.println("Result is " + dice);
    }
}
```

5.Method

Declaring a method:

```
public static int addTwoNumbers() {
int result = 3 + 5;
return result;
```

} 有返回值 int 结果可以使用

```
public static void addTwoNumbers() {
int result = 3 + 5;
System.out.println(result);
return;
```

} 无返回值 仅在屏幕上打印 不能使用

Calling a method multiple times

```
public static void main(String[] args) {
    int total = addTwoNumbers(3, 5);
    displayTotal(total);
    total = addTwoNumbers(8, 2);
    displayTotal(total);
}
public static int addTwoNumbers(int num1, int num2) {
      int result = num1 + num2;
      return result;
}
```

Same method called two times, with different arguments

Different totals each time

Parameters passed in

生成色子+求和+main 方法

```
public static int[] generateDice(int[] input) {
    // Loop for length of array
    for (int i = 0; i < input.length; i++) {
        // Dice generation code
        double rand =
                  ((Math.random() * 6) + 1);
        int dice = (int) rand;
        input[i] = dice;
        System.out.println("Result is " +
                           dice);
    }
    // return the array
    return input;
}
```

```
public static int findTotal(int[] input){
    int total = 0;
    for (int i = 0; i < input.length; i++) {
        total = total + input[i];
        System.out.println("This dice value
                                  is " + input[i]);
    }
    return total;
}
```

```
public static void main(String[] args) {
    int[] diceList = new int[4];

    diceList = generateDice(diceList);

    int total = findTotal(diceList);
    System.out.println("The total is " +
                                   total);
}
```

Printing All values Finding Max Value Finding Max Index

Lab 4

1. Max Consecutive Sum Exe41

```
//遍历数组中每一个数字 前几个的和与当前数字作比较 当前数字大就从当前的开始
int currentsum = input[0];
//保存最大
int maxsum = input[0];
for (int i = 1;i<input.length;i++){
    currentsum += input[i];
    currentsum = Math.max(currentsum,input[i]);
    maxsum = Math.max(currentsum,maxsum);    //当前的sum与历史最大sum比较
}
return maxsum;
```

2. **Sum of Proper Elements** sumOfProperElements

3.Max Product CW1

```
int maxProduct = Integer.MIN_VALUE;

for(int i=0;i<arr.length;i++){
    for(int j=i+1;j<arr.length;j++){
    //把每一对都乘一遍 然后比较
        int product = arr[i]*arr[j];
        if(product>maxProduct){
            maxProduct = product;
```

Lec5 Static Methods 2 and String

1.输入输出数量关系

`Math.random()` 0 input, 1 return value

`Integer.parseInt()` 1 input, 1 return value

2 input, 1 return value

2.`Method signature` of a method consists of

- method name
- parameter types

3.Overloading 参数不同

编译器行为： 编译器在编译时（Compile time）就能确定调用哪一个

Overwritting 参数相同

编译器行为： 运行时（Runtime）才确定调用谁的方法（多态）。

4. arrays and String, is not a primitive type but rather a reference type(引用类型), we will see later that when we declare a reference type in a variable, we store not the data, but the reference / address / pointer of the data

5.

boolean isLetter(char c) is c a letter?

boolean isDigit(char c) is c a digit?

boolean isWhitespace(char c) is c white space?

boolean isUpperCase(char c) is c an uppercase?

boolean isLowerCase(char c) is c a lowercase?

```
char firstLetter = 'A';
char five = '5';
char newLine = '\n';
Character.isLetter(firstLetter);
Character.isDigit(five);
Character.isWhitespace(newLine);
Character.isUpperCase(firstLetter);
Character.isLowerCase(firstLetter);
```

`char toUpperCase(char c)` to uppercase

`char toLowerCase(char c)` to lowercase

`String toString(char c)` to a `String` object consist of one character

```
Character.toUpperCase(smallA);
Character.toLowerCase(firstLetter);
String oneLetterStr = Character.toString(firstLetter);
```

只能转变单个字符

```
char[] letters = {'H', 'e', 'l', 'l', 'o'};
String s = new String(letters); // 或者
String s2 = String.valueOf(letters);
```

6.Length substring()

```
String cosmo = "Halloween";
int len = cosmo.length();
```

`substring(int start)` returns a new string made of the chars starting at index `start` and continue until the end of the string

(从 start 开始到最后一个字符结束)

```
String cosmo = "Halloween";
```

```
String ss1 = cosmo.substring(1); //alloween
```

(int start, int end)notice the length of the resulting substring can be computed by subtracting end - start

```
String ss1 = cosmo.substring(2, 5); //llo
```

7.bound errors:数组越界问题 Runtime error

Check :

```
if (str.length() >= 5) {
prefix = str.substring(0, 5);
}
else {
// do something else when length is < 5
}
```

8.`cosmo.equals("Halloween")`

Use the `equals()` method to check if 2 strings are the same `equals()` method is case-sensitive 区分大小写 不能用==

9.String Test

`boolean isEmpty()`	is string an empty string?
`boolean contains(String substring)`	does string contain `substring`?
`boolean startsWith(String prefix)`	does string start with `prefix`?
`boolean endsWith(String postfix)`	does string end with `postfix`?

```
cosmo.isEmpty();

cosmo.contains("lowe");

cosmo.startsWith("hal");

cosmo.endsWith("ween");
```

```
String toUpperCase() to uppercase
String toLowerCase() to lowercase
String cosmo = "Halloween";
cosmo.toUpperCase();
cosmo.toLowerCase();
```

10. CharAt(i)

```
for (int i = 0; i < str.length(); i++) {
// do something to str at index i
System.out.println(str.charAt(i));
}
```

11. Indexof

`str.indexOf(String target)` method searches left-to-right inside `str` for the string `target`, it returns the index number where the target string is first found, or `-1` if the target is not found

case-sensitive

```
String cosmo = "Halloween";
int a = cosmo.indexOf("allo");
int b = cosmo.indexOf("e");
int c = cosmo.indexOf("allO");  // 1,6,-1
```

There is also `lastIndexOf(String target)` method that searches for the `target` from right to left

```
int d = cosmo.lastIndexOf("e");
```

12. indexOf, lastIndexOf with fromIndex

`int indexOf(String target, int fromIndex)` : left-to-right, starts the search at the given `fromIndex`
`int lastIndexof(String target, int fromIndex)` : right-to-left, beginning at the given `fromIndex`

`fromIndex` does not actually need to be valid

if it is negative, the search happens <u>from the start of</u> the string

if it is greater than the string length, then -1 will be returned

13.Find all instances

```
public static void findHallo(String str) {
    int start = 0;
    while (true) {
        int found = str.indexOf("Hallo", start);
        if (found != -1) {
            // found one, do something here
        }
        if (found == -1) break; // found none, need to stop looping
        start = found + 2;      // move start up for next iteration
    }
}
```

文本框字符大写：

```
public static String capitalParen(String str) {

    int left = str.indexOf("(");
    int right = str.indexOf(")");

    String sub = str.substring(left + 1, right);

    sub = sub.toUpperCase();

    String result = str.substring(0, left + 1) + sub +
            str.substring(right);

    return result;
}
```

14.Trim Split Replace

`String trim()` this string with leading and trailing whitespace removed
`String replace(String a, String b)` this string with a replaced by b
`String replaceAll(String a, String b)` this string with all a's replaced by b's
`String[] split(String delimiter)` array of strings between occurrences of `delimiter`

```
String cosmo = " ha haa halloween ";
cosmo = cosmo.trim();
cosmo = cosmo.replaceAll("h", "H");
String[] words = cosmo.split(" ");
for (int i = 0; i < words.length; i++) {
System.out.println(words[i]);
}
```

Must be 13 characters long

Must be 4 blocks

Must be separated by hyphens

First character must be between A and D

Other 3 blocks are 3 characters long

2nd and 3rd blocks are all numbers

Final block must be 2 numbers and a letter

The letter must be between A and T

15.

```
if (id == null || id.length() != 13) { return false; }
if (id.charAt(1) != '-' || id.charAt(5) != '-' || id.charAt(9) != '-')
{ return false; }
char firstChar = id.charAt(0); if (firstChar < 'A' || firstChar > 'D')
{ return false; }
if (!isDigit(id.charAt(2)) || !isDigit(id.charAt(3))
|| !isDigit(id.charAt(4))) { return false; }
char lastChar = id.charAt(12);
if (lastChar < 'A' || lastChar > 'T') { return false; }
```

Lab5

1. War peace 字符串中单词出现次数 Lab51

Post1=Indexof(“ ”):str 第一次出现的位置

Post1= Indexof(str, fromIndex) 第一次出现后下一个

有目标字符的，用 indexof()确定首位，+length()截取

`index=input.indexOf(target))!=-1` 循环判断

2.removeLeftmostSubstring challenge

```
int index;
//.indexOf(target)
 while((index=input.indexOf(target))!=-1){
     input= input.substring(0,index)+input.substring( beginIndex: index+target.length());
 }
```

3.

Lec6

1. **核心概念与基础**（Theory）

OOP 核心思想: 把世界看作对象，对象包含状态 (State/Variables) 和行为 (Behavior/Methods) - **Page 10-12**

Primitive vs Object

基本类型 (int, double)：只存值，没有方法 - **Page 7**

对象类型 (String, Scanner)：存地址引用，有方法 - **Page 9**

Wrapper Classes (包装类): 把基本类型当对象用 (如 Integer.parseInt) - **Page 8**

2. **类的定义**（Class Structure）

实例变量 (Instance Variables): 定义在类里，没有 static，每个人一份 - **Page 26Controller Class (控制类)**: 包含 main 方法的类，用来运行程序 - **Page 47**

Data Class (数据类): 像 Person 这种，只有数据和构造器，通常没有 main - **Page 47**

3. **构造器**（Constructors）—— ★ **必考**

定义规则:名字必须**与类名完全一致** (大小写也要一样) - **Page 32**

没有返回类型 (连 void 都没有) - **Page 32**

作用: 初始化对象，给实例变量赋初值 - **Page 29-30**

参数传递: 如何把 main 里的数据传进对象里 - **Page 33**

4. `this` 关键字

作用: 指代“当前对象” - **Page 42**

解决命名冲突: 当构造器参数名 (local) 和实例变量名 (instance) 一样时，必须用 this.name = name - **Page 41**

5. **封装**（Encapsulation）& **访问控制**

Public vs Private:

Private: 只有自己类能看（用于数据安全，变量通常设为 private- **Page 52-53**

Public: 大家都能看（用于构造器和方法） - **Page 52-53**

Getter Methods (访问器): getName()，用来把 private 的数据读出来 - **Page 54-56**

Immutable Objects (不可变对象): 现在的趋势是只给 Getter 不给 Setter，一旦创建不能修改 - **Page 59**

6. Static vs Non-Static —— ★重难点

Static (静态):

属于**整个类**，只有一份拷贝 - **Page 60**

如果在 Person 名字上用 Static，所有人都会叫同一个名字（后一个覆盖前一个） - **Page 61**

适用场景: main 方法，或者计数器 (count objects) - **Page 64**

Non-Static (非静态/实例):

属于**具体对象**，每个对象各有一份 - **Page 62**

7. 对象比较与内存 (Memory & Comparison)

引用传递 (Pass by Reference): 对象变量存的是**内存地址** (Pointer)，不是值 - **Page 69**

== 的坑

基本类型比数值 (True) - **Page 73**

对象比地址 (通常 False，除非指向同一个内存) - **Page 73**

自定义比较方法 (samePerson): 如何写一个方法对比两个对象的属性 (name, id) - **Page 77-81**

7. 输出与数组 (Output & Arrays)

`toString()` 方法:

默认打印内存地址（很丑）- **Page 83**

如何重写（Override）来打印漂亮的信息 - **Page 84**

对象数组: Person[] people = new Person[3]; 以及如何用循环遍历 - **Page87-89**

Lec8

1. Static Instance 变量
2. Static 方法 构造器重载
3. 继承 final
4. 方法重写 overriding
5. 多态

子类构造器：public Pillar(String name) { super(name); } // P28

调用父类逻辑：return 100 + super.attackDamage(); // P40

多态声明：Swordsman s = new Pillar("Kyojuro"); // P47

定义常量：public static final int MAX_HP = 100; // P34

1. List (ArrayList / LinkedList) - *有序列表*

方法	功能	代码示例
`add(E e)`	添加元素到末尾	`list.add("apple");`
`get(int index)`	获取指定位置的元素	`String s = list.get(0);`
`size()`	获取列表长度	`int len = list.size();`
`contains(Object o)`	判断是否包含某元素	`if (list.contains("apple"))`
`remove(int index)`	删除指定位置的元素	`list.remove(0);`
`isEmpty()`	判断是否为空	`if (list.isEmpty())`

导出到 Google 表格

2. Set (HashSet) - *去重集合*

方法	功能	代码示例
`add(E e)`	添加元素 (重复的会被忽略)	`set.add("apple");`
`contains(Object o)`	**核心：** 快速判断是否存在	`if (set.contains("apple"))`
`size()`	获取集合大小	`int size = set.size();`
`remove(Object o)`	删除指定元素	`set.remove("apple");`
`clear()`	清空集合	`set.clear();`

导出到 Google 表格

3. Map (HashMap) - *键值对 (字典)*

方法	功能	代码示例
`put(K key, V value)`	放入/更新数据	`map.put("apple", 5);`
`get(Object key)`	根据 Key 获取 Value	`int count = map.get("apple");`
`containsKey(Object key)`	**核心：** 判断 Key 是否存在	`if (map.containsKey("apple"))`
`keySet()`	**核心：** 获取所有 Key (用于遍历)	`for (String k : map.keySet())`
`remove(Object key)`	根据 Key 删除整条记录	`map.remove("apple");`

导出到 Google 表格

Array (数组)

语法	说明	示例
`int[] arr = new int[5];`	创建指定长度数组	默认值为 0
`int[] arr = {1, 2, 3};`	初始化数组	直接赋值
`arr.length`	**获取长度 (无括号)**	`for (int i=0; i<arr.length; i++)`
`arr[i]`	访问元素	`int x = arr[0];`
`Arrays.sort(arr);`	排序 (从小到大)	需要 `import java.util.Arrays;`
`Arrays.toString(arr)`	打印数组内容	调试用，打印 `[1, 2, 3]`

导出到 Google 表格

String (字符串)

方法	说明	示例
`str.length()`	**获取长度 (有括号)**	`int len = str.length();`
`str.charAt(i)`	获取指定字符	`char c = str.charAt(0);`
`str.substring(start, end)`	截取子串	`str.substring(1, 4)` (含1不含4)
`str.indexOf("text")`	查找子串位置	返回索引，没找到返回 -1
`str.split(" ")`	分割字符串	`String[] parts = str.split(",");`
`str.trim()`	去除首尾空格	`str = str.trim();`
`str.equals(s2)`	**比较内容 (千万别用 ==)**	`if (s1.equals(s2))`
`Integer.parseInt(str)`	字符串转数字	`int n = Integer.parseInt("123");`

`str.substring(start, end)`	截取子串	`str.substring(1, 4)` (含1不含4)
`str.indexOf("text")`	查找子串位置	返回索引，没找到返回 -1
`str.split(" ")`	分割字符串	`String[] parts = str.split(",");`
`str.trim()`	去除首尾空格	`str = str.trim();`
`str.equals(s2)`	**比较内容 (千万别用 ==)**	`if (s1.equals(s2))`
`Integer.parseInt(str)`	字符串转数字	`int n = Integer.parseInt("123");`

| `Character.isDigit(c)` | 判断字符是否为数字 | `if (Character.isDigit(c))` |
| `Character.isLetter(c)` | 判断字符是否为字母 | `if (Character.isLetter(c))` |